

Lab 06 Characters, Strings, and Mathematical Functions

Objective

The purpose of this lab is to familiarize students with characters and strings data types and how to manipulate characters and string values using methods of characters and strings. It also provides additional mathematical functions. There are some exercises, through which they will understand the concept learned in this chapter.

Current Lab Learning Outcomes (LLO)

By completion of the lab the students should be able to

1. Learn how to write Java programs that uses data types of char and String.
2. Solve different problems using appropriate methods of Character class and instances methods of references variables of type String.
3. Solve mathematical problems using appropriate function from the class Math.

Lab Requirements

NetBeans IDE 8.0.1 or higher

Lab Assessment/Activities

Practice Activity with Lab Instructor (10 minutes)

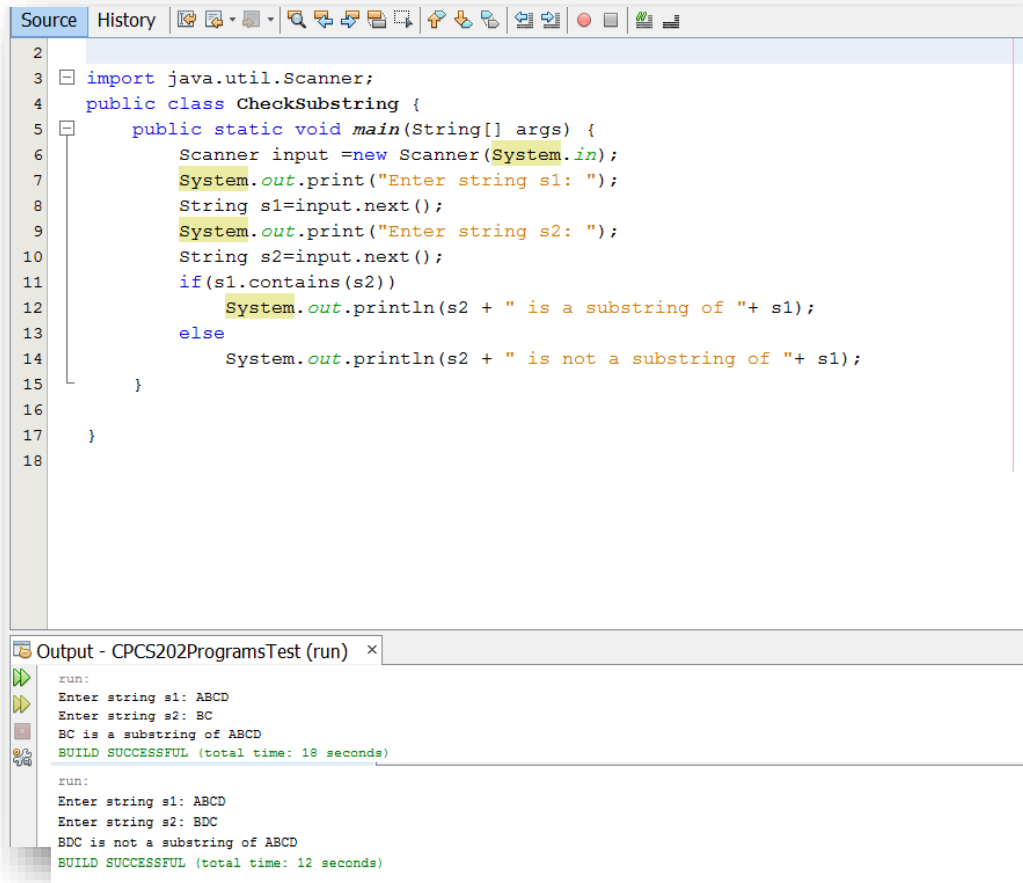
(Check substring) Write a program that prompts the user to enter two strings and reports whether the second string is a substring of the first string.

```
Enter string s1: ABCD Enter
Enter string s2: BC Enter
BC is a substring of ABCD
```

```
Enter string s1: ABCD Enter
Enter string s2: BDC Enter
BDC is not a substring of ABCD
```

Solution:

1. Open NetBeans and create a new project
2. Create a new java main class and write its name as **CheckSubstring**
3. Write the following code inside the main method, Figure (1).



```
Source History
2
3 import java.util.Scanner;
4 public class CheckSubstring {
5     public static void main(String[] args) {
6         Scanner input = new Scanner(System.in);
7         System.out.print("Enter string s1: ");
8         String s1 = input.next();
9         System.out.print("Enter string s2: ");
10        String s2 = input.next();
11        if (s1.contains(s2))
12            System.out.println(s2 + " is a substring of " + s1);
13        else
14            System.out.println(s2 + " is not a substring of " + s1);
15    }
16
17 }
18
```

Output - CPCS202ProgramsTest (run) ×

run:
Enter string s1: ABCD
Enter string s2: BC
BC is a substring of ABCD
BUILD SUCCESSFUL (total time: 18 seconds)

run:
Enter string s1: ABCD
Enter string s2: BDC
BDC is not a substring of ABCD
BUILD SUCCESSFUL (total time: 12 seconds)

Figure 1: The program to check if one string is a substring from the other

Short Exercise (5 minutes)

The above program is **case sensitive**, rewrite it such that become **case insensitive** and display the position of second string inside the first string if it is a substring of it.

Individual Activities: (60 minutes)

1. **(Math Methods)** Write a program that generates two random double numbers **between -20 and 20** and displays the maximum number after finds the absolute values for negative numbers.
2. **(Vowel or consonant?)** Write a program that prompts the user to enter a letter and check whether the letter is a vowel or consonant. **Here is a sample run:**

```
Enter a letter: B ↵ Enter  
B is a consonant
```

```
Enter a letter grade: a ↵ Enter  
a is a vowel
```

```
Enter a letter grade: # ↵ Enter  
# is an invalid input
```

3. **(Student major and status)** Write a program that prompts the user to enter two characters and displays the major and status represented in the characters. The first character indicates the major and the second is number character 1, 2, 3, 4, which indicates whether a student is a freshman, sophomore, junior, or senior. Suppose the following characters are used to denote the majors:

M: Mathematics

C: Computer Science

I: Information Technology

Here is a sample run:

```
Enter two characters: M1 ↵ Enter  
Mathematics Freshman
```

```
Enter two characters: C3 ↵ Enter  
Computer Science Junior
```

```
Enter two characters: T3 ↵ Enter  
Invalid input
```

4. **(Check SSN)** Write a program that prompts the user to enter a Social Security number in the format DDD-DD-DDDD, where D is a digit. Your program should check whether the input is valid. **Here are sample runs:**

```
Enter a SSN: 232-23-5435 [Enter]
232-23-5435 is a valid social security number
```

```
Enter a SSN: 23-23-5435 [Enter]
23-23-5435 is an invalid social security number
```

5. **(Order three cities)** Write a program that prompts the user to enter three cities and displays them in ascending order. Here is a sample run:

```
Enter the first city: Chicago [Enter]
Enter the second city: Los Angeles [Enter]
Enter the third city: Atlanta [Enter]
The three cities in alphabetical order are Atlanta Chicago Los Angeles
```

Lab Description

Theory Review (5 minutes)

- Java provides many useful methods in the **Math** class for performing common mathematical functions. There are many beneficial methods inside the Math class. **(See the appendix below).**

The char Type

- Declaring variable of **char** type

```
char letter = 'A';
```

```
char letter = '\u0041'; // A's Unicode value is 0041
```

Both statements assign character **A** to the **char** variable **letter**

- Reading a Character from the Console

The following statements use to read the value from keyboard to the variable of type, **char**, **ch**.

```
Scanner input = new Scanner(System.in);
```

```
System.out.print("Enter a character: ");
```

```
char ch= input.nextLine().charAt(0);
```

- Java provides many methods inside the **Character** class. **(See the appendix below).**

The String Type

- The `char` type represents only one character. To represent a string of characters, use the data type called **String** which is a predefined class in the Java library.

- Declaring a variable of **String** type

```
String name;
```

- Assign a value to the **String** variable

```
name= "Muhammad Alzahrani";
```

Or, you can combine the declaration and assignment statements as follows:

```
String name= "Muhammad Alzahrani";
```

Reading a String from the Console

The following statements use to read the value from keyboard to the variable of type, **String**, **str**.

```
Scanner input = new Scanner(System.in);  
System.out.println("Enter a line: ");  
String str = input.nextLine();
```

- Java provides many of methods that operate on Strings. (See the appendix below)

Formatting Output

- You can use the `System.out.printf` method to display formatted output. When printing double values, often we do not need or want all the decimals. In fact, often we want only two (for money)!

Example: Displaying 2 digits after decimal point

```
double x = 16.404674;
```

```
System.out.printf("x is %4.2f", x);
```

The Output will be : **x is 16.40**

- Common format specifiers are shown in Figure (2).

Frequently Used Format Specifiers		
Format Specifier	Output	Example
%b	a Boolean value	true or false
%c	a character	'a'
%d	a decimal integer	200
%f	a floating-point number	45.460000
%e	a number in standard scientific notation	4.556000e+01
%s	a string	"Java is cool"

Figure 2: The common format specifiers

Extra (supplementary) Materials (if any)

None

Appendix

[Some helpful methods including mathematical, characters and string manipulation methods:](#)

Exponent Methods in the Math Class	
Method	Description
<code>exp(x)</code>	Returns e raised to power of x (e^x).
<code>log(x)</code>	Returns the natural logarithm of x ($\ln(x) = \log_e(x)$).
<code>log10(x)</code>	Returns the base 10 logarithm of x ($\log_{10}(x)$).
<code>pow(a, b)</code>	Returns a raised to the power of b (a^b).
<code>sqrt(x)</code>	Returns the square root of x ($\sqrt{x}$) for $x \geq 0$.

Figure 3: The exponent methods in the Math class

- **`double ceil(double x)`**
x rounded up to its nearest integer. This integer is returned as a double value.
- **`double floor(double x)`**
x is rounded down to its nearest integer. This integer is returned as a double value.
- **`double rint(double x)`**
x is rounded to its nearest integer. If x is equally close to two integers, the even one is returned as a double.
- **`int round(float x)`**
Return (int)Math.floor(x+0.5).
- **`long round(double x)`**
Return (long)Math.floor(x+0.5).

Figure 4: The rounding methods in the Math class

☞ **`min`** and **`max`** methods

- **`min(a, b)`** : returns minimum of a and b
- **`max(a, b)`** : returns the maximum of a and b

☞ **`abs`** method:

- **`abs(a)`** : returns the absolute value of a

Figure 5: The min, max, abs, and random methods

Methods in the Character Class

Method	Description
<code>isDigit(ch)</code>	Returns true if the specified character is a digit.
<code>isLetter(ch)</code>	Returns true if the specified character is a letter.
<code>isLetterOfDigit(ch)</code>	Returns true if the specified character is a letter or digit.
<code>isLowerCase(ch)</code>	Returns true if the specified character is a lowercase letter.
<code>isUpperCase(ch)</code>	Returns true if the specified character is an uppercase letter.
<code>toLowerCase(ch)</code>	Returns the lowercase of the specified character.
<code>toUpperCase(ch)</code>	Returns the uppercase of the specified character.

Figure 6: The methods in the Character class

Simple Methods for **String** Objects

Method	Description
<code>length()</code>	Returns the number of characters in this string.
<code>charAt(index)</code>	Returns the character at the specified index from this string.
<code>concat(s1)</code>	Returns a new string that concatenates this string with string s1.
<code>toUpperCase()</code>	Returns a new string with all letters in uppercase.
<code>toLowerCase()</code>	Returns a new string with all letters in lowercase
<code>trim()</code>	Returns a new string with whitespace characters trimmed on both sides.

Figure 7: Simple methods for String object

Comparison Methods for **String** Objects

Method	Description
<code>equals(s1)</code>	Returns true if this string is equal to string s1.
<code>equalsIgnoreCase(s1)</code>	Returns true if this string is equal to string s1; it is case insensitive.
<code>compareTo(s1)</code>	Returns an integer greater than 0, equal to 0, or less than 0 to indicate whether this string is greater than, equal to, or less than s1.
<code>compareToIgnoreCase(s1)</code>	Same as <code>compareTo</code> except that the comparison is case insensitive.
<code>startsWith(prefix)</code>	Returns true if this string starts with the specified prefix.
<code>endsWith(suffix)</code>	Returns true if this string ends with the specified suffix.
<code>contains(s1)</code>	Returns true if s1 is a substring in this string.

Figure 8: Comparison methods for String object

The **String** class contains the methods for finding substrings.

Method	Description
<code>indexOf(ch)</code>	Returns the index of the first occurrence of ch in the string. Returns -1 if not matched.
<code>indexOf(ch, fromIndex)</code>	Returns the index of the first occurrence of ch after fromIndex in the string. Returns -1 if not matched.
<code>indexOf(s)</code>	Returns the index of the first occurrence of string s in this string. Returns -1 if not matched.
<code>indexOf(s, fromIndex)</code>	Returns the index of the first occurrence of string s in this string after fromIndex. Returns -1 if not matched.
<code>lastIndexOf(ch)</code>	Returns the index of the last occurrence of ch in the string. Returns -1 if not matched.
<code>lastIndexOf(ch, fromIndex)</code>	Returns the index of the last occurrence of ch before fromIndex in this string. Returns -1 if not matched.
<code>lastIndexOf(s)</code>	Returns the index of the last occurrence of string s. Returns -1 if not matched.
<code>lastIndexOf(s, fromIndex)</code>	Returns the index of the last occurrence of string s before fromIndex. Returns -1 if not matched.

Figure 9: The methods of finding substrings in the String class